Planificación de Horarios Universitarios: revisión del estado del arte y estudio de reproducibilidad en CB-CTT

Jorge Martin La Peña, Alejandro Moris Lara y Jose Miguel Villacorta Arteaga

Resumen. La planificación de horarios universitarios asigna sesiones docentes a periodos y aulas bajo restricciones obligatorias y criterios de calidad. Este trabajo revisa los métodos publicados entre 2015 y 2026 para el problema de planificación basado en grupos de asignaturas (CB-CTT). También estudia la reproducibilidad de un método reciente sobre 19 instancias de la tercera modalidad de la International Timetabling Competition 2007 (ITC-2007). La revisión reúne 25 estudios y compara métodos exactos, metaheurísticas, enfoques híbridos, hiperheurísticas y técnicas de selección automática. La reproducción se ejecutó con una búsqueda local que combina enfriamiento simulado y el vecindario LectureKick. El estudio permite comprobar el funcionamiento del método y valorar sus diferencias respecto al artículo original. La revisión registra cómo se publican los datos, el código y las configuraciones experimentales.

Palabras clave. planificación de horarios universitarios, CB-CTT, ITC-2007, metaheurísticas, optimización combinatoria, reproducibilidad, LectureKick.

Abstract. University course timetabling assigns teaching activities to periods and rooms while satisfying mandatory constraints and minimizing quality penalties. This paper reviews methods published between 2015 and 2026 for the Curriculum-Based Course Timetabling Problem (CB-CTT). It also studies the reproducibility of a recent method on 19 instances from the third track of the International Timetabling Competition 2007 (ITC-2007). The review covers 25 studies and compares exact methods, metaheuristics, hybrid approaches, hyper-heuristics, and automatic algorithm selection. Reproducibility was assessed using a local search that combines simulated annealing with the LectureKick neighborhood. The study checks the method and highlights its differences from the original article. The review records how benchmark data, source code, and experimental configurations are reported.

Index Terms. university course timetabling, CB-CTT, ITC-2007, metaheuristics, combinatorial optimization, reproducibility, LectureKick.

I. Problema de investigación

La planificación de horarios universitarios exige coordinar sesiones docentes, profesorado y aulas con criterios de calidad. Este trabajo estudia CB-CTT y reproduce un método reciente de búsqueda local sobre la formulación UD2 de la tercera modalidad de la International Timetabling Competition 2007 (ITC-2007).

En CB-CTT se asigna un periodo y un aula a cada sesión de una semana lectiva, se separan las sesiones de asignaturas que comparten alumnado y se evitan coincidencias de aula, profesor e indisponibilidad. Estas condiciones son restricciones duras, por lo que un horario solo es factible cuando las satisface todas [1].

Máster Universitario en Aprendizaje Automático y Datos Masivos, Universidad Politécnica de Madrid.

Tabla I
Penalizaciones de la tercera modalidad de ITC-2007. Fuente: [1].

Criterio	Penalización en la formulación UD2
Capacidad	1 punto por estudiante sin plaza y sesión.
Días mínimos de docencia	5 puntos por día faltante respecto al mínimo de la asignatura.
Compactación del grupo	2 puntos por sesión aislada y grupo, sin sesión adyacente del mismo grupo ese día.
Estabilidad del aula	1 punto por aula distinta adicional utilizada por una asignatura.

La calidad de los horarios factibles se evalúa mediante la suma ponderada de cuatro penalizaciones. En UD2, la capacidad del aula se considera una restricción blanda, de modo que superar sus plazas aumenta el coste, pero no invalida por sí solo el horario [1].

II. Objetivos y preguntas de investigación

El objetivo general es describir la evolución reciente de los métodos para CB-CTT y comprobar hasta qué punto puede reproducirse un resultado reciente con materiales disponibles públicamente. Para ello, la tercera modalidad de ITC-2007 constituye la referencia experimental principal e ITC-2019 aporta el contexto necesario para comentar la evolución hacia formulaciones universitarias más completas.

Las preguntas de investigación son las siguientes:

1) ¿Qué enfoques algorítmicos se han utilizado para CB-CTT entre 2015 y 2026?
2) ¿Qué características tienen los conjuntos de instancias y qué papel conserva ITC-2007?
3) ¿Qué tendencias aparecen en hibridación, hiperheurísticas, selección automática y aprendizaje automático?
4) ¿Hasta qué punto pueden reproducirse resultados recientes con implementaciones y datos públicos?

III. Búsqueda y selección bibliográfica

La revisión bibliográfica abarcó el periodo comprendido entre 2015 y 2026. Para localizar estudios adicionales se combinaron páginas editoriales, repositorios institucionales y de autores, recursos oficiales de los conjuntos de instancias y las revisiones de Bettinelli et al. [3], Ceschia et al. [2] y Gu et al. [4].

De cada estudio se registraron el año, el método, el conjunto de instancias, las métricas, los comparadores y la disponibilidad de datos y código, junto con la información necesaria para repetir el experimento.

IV. Benchmarks de referencia

La tercera modalidad de ITC-2007 proporciona 21 instancias de la Universidad de Udine, publicadas en tres conjuntos de siete, y sirve como referencia porque utiliza una definición común de restricciones y coste [1]. Ceschia et al. [2] excluyen dos instancias del conjunto: una tiene coste objetivo cero (comp11) y la otra es equivalente a comp03 en la formulación UD2 (comp15). Por esa razón, este trabajo utiliza las 19 instancias restantes.

ITC-2019 responde a un modelo más amplio que integra la asignación de clases a horarios y aulas con la distribución de estudiantes entre grupos, conocida como *"student sectioning"*. También contempla configuraciones de teoría, prácticas o laboratorios, patrones de días y semanas y restricciones de distribución. La descripción de sus organizadores documenta 30 instancias competitivas y seis de prueba procedentes de diez universidades [5].

La función objetivo de ITC-2019 combina penalizaciones de tiempo, aula, distribución y conflictos de estudiantes, con pesos definidos en cada instancia [5]. Sus puntuaciones no se pueden comparar directamente con el coste de UD2 porque ITC-2007 representa los conflictos mediante grupos de asignaturas, mientras que ITC-2019 añade la asignación de estudiantes a grupos. Cualquier comparación debe indicar, por tanto, la formulación y el conjunto de instancias utilizado.

V. Estado del arte

A. Métodos exactos y basados en restricciones

1) Programación entera y descomposición: La programación lineal entera mixta (MIP o MILP) modela las asignaciones mediante variables binarias y expresa las restricciones como relaciones algebraicas. La búsqueda por ramificación y acotación, reforzada con cortes válidos, puede generar horarios y cotas inferiores [3]. En un problema de minimización, un horario factible fija una cota superior, mientras que la cota inferior marca un valor que el óptimo no puede rebasar por abajo. Si ambas coinciden, se certifica la optimalidad. Cuando el tiempo se agota antes, la mejor solución encontrada debe describirse como factible, no como óptima.

La formulación también condiciona el rendimiento. Bagger, Kristiansen, Sørensen y Stidsen [6] conectan la planificación temporal y la asignación de aulas mediante modelos de flujo. En sus experimentos, la cota superior media de esta formulación supera a la de las alternativas MIP comparadas, aunque el resultado se limita a ese conjunto de instancias y no establece una ventaja general para todas las familias de algoritmos.

Bagger, Sørensen y Stidsen [7] aplican descomposición de Dantzig-Wolfe y generación de columnas a patrones diarios. En las 20 instancias de su estudio mejoran las cotas inferiores de cuatro casos y reducen la brecha media del 24 % al 11 %. Esta mejora aproxima mejor el óptimo, pero no equivale a reducir en la misma medida el coste de los horarios. El procedimiento calcula cotas, pero el artículo no presenta un algoritmo completo de branch-and-price.

2) Restricciones y modelos lógicos: La programación por restricciones (CP) trabaja con dominios de valores y propaga restricciones para descartar asignaciones incompatibles. La satisfacibilidad proposicional (SAT) formula la factibilidad mediante cláusulas lógicas, mientras que MaxSAT parcial ponderado permite optimizar penalizaciones sin relajar las cláusulas obligatorias. Asín Achá y Nieuwenhuis [8] constituyen un antecedente de estas técnicas en CB-CTT, cuya diferencia entre satisfacer restricciones y optimizar el coste resulta esencial para interpretar los métodos recientes.

La programación de conjuntos de respuesta (ASP) ofrece otro enfoque declarativo. Banbara et al. [9] desarrollan teaspoon y evalúan 61 instancias con cinco formulaciones, es decir, 305 combinaciones, en las que informan de 54 mejoras sobre resultados previos, incluidas 16 nuevas soluciones óptimas. Estas cifras corresponden a las formulaciones y configuraciones de su estudio y no solo a las 21 instancias originales de ITC-2007 UD2.

El análisis distingue tres aportaciones: obtener un horario factible, establecer una cota inferior y certificar un óptimo. Mantenerlas separadas evita interpretar una mejora de cota como una mejora equivalente del horario producido.

B. Metaheurísticas

Una vez alcanzada la factibilidad, todavía quedan muchas combinaciones posibles, por lo que buena parte de la literatura de CB-CTT recurre a metaheurísticas y búsqueda local [2]. Estas técnicas exploran vecindarios de horarios y combinan reglas de aceptación con mecanismos de diversificación para salir de óptimos locales.

Los primeros trabajos sobre CB-CTT combinaron movimientos de reasignación e intercambio con Hill Climbing y búsqueda tabú. En la Segunda Competición Internacional de Horarios de 2007 (Second International Timetabling Competition, ITC-2007), el método de Müller obtuvo el mejor resultado en la modalidad dedicada a este problema. Su propuesta coordina cinco tipos de movimiento con Hill Climbing, Gran Diluvio (Great Deluge) y enfriamiento simulado [10]. Hill Climbing conserva los cambios que mejoran el horario, Great Deluge aplica un umbral de aceptación variable y el enfriamiento simulado permite aceptar algunos empeoramientos.

En un problema de minimización, el enfriamiento simulado acepta siempre una mejora y admite un empeoramiento de magnitud Δ con probabilidad $\exp(-\Delta/T)$. La temperatura T disminuye durante la ejecución. Al principio se aceptan más cambios desfavorables para salir de horarios que solo parecen buenos en su entorno inmediato, mientras que al final la búsqueda se concentra en reducir el coste. La Fig. 1 resume este ciclo, en el que cada solución nueva es un horario vecino y la aceptación depende de su penalización y de la temperatura.

Bellio et al. aplicaron este esquema a CB-CTT y mostraron que la temperatura inicial influye en la calidad final [12], mientras que Song et al. incorporaron varios tipos de movimiento y ajustaron su frecuencia de selección [13]. Cuando hay varios movimientos, la búsqueda debe decidir cuál aplicar en

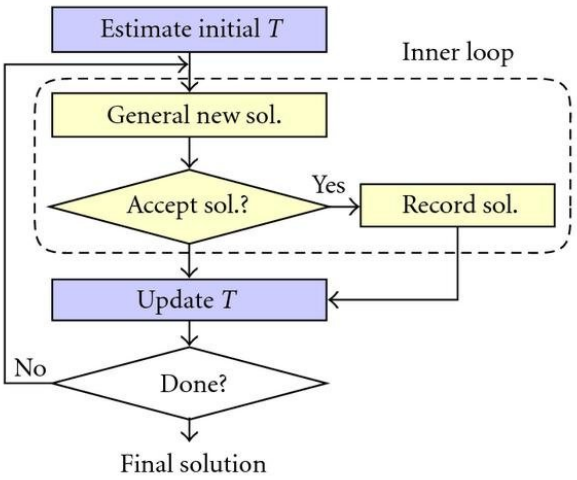


Figura 1. Ciclo general del enfriamiento simulado. Para CB-CTT, la nueva solución corresponde a un horario vecino y la temperatura regula la aceptación de los cambios que aumentan su penalización. Reproducido de la Fig. 1 de Ababei [11].

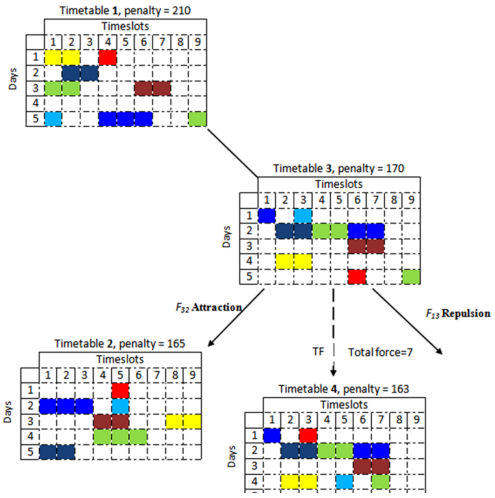


Figura 2. Ejemplo del componente de atracción y repulsión del método híbrido de Abdullah et al. Los horarios con penalizaciones 210 y 165 modifican la búsqueda iniciada desde el horario con penalización 170, que conduce a una nueva solución con penalización 163. Reproducido de la Fig. 2 de Abdullah et al. [14].

cada momento. La búsqueda de vecindario variable (Variable Neighborhood Search, VNS) cambia de vecindario cuando el actual deja de mejorar, y el descenso por vecindarios variables (Variable Neighborhood Descent, VND) sigue una secuencia fija de búsquedas locales. Las hiperheurísticas sitúan esta decisión en un nivel superior y seleccionan el movimiento de bajo nivel.

Otros métodos amplían la búsqueda manteniendo varios horarios a la vez. Abdullah et al. combinan una población de soluciones con un mecanismo inspirado en el electromagnetismo y Great Deluge [14], una combinación que anticipa el uso de métodos híbridos en planificación universitaria recogido por Abdipoor et al. [15].

C. Métodos híbridos

Los métodos híbridos combinan técnicas que intervienen en momentos distintos de la búsqueda. Un algoritmo memético genera varios horarios y aplica una búsqueda local para mejorar algunos de ellos, mientras que una matheurística retira temporalmente un conjunto de clases y formula su recolocación como un problema matemático. Kiefer et al. siguen este segundo enfoque mediante una búsqueda adaptativa de gran vecindario (Adaptive Large Neighborhood Search, ALNS) [16]. La parte adaptativa decide qué clases retirar según el rendimiento reciente de cada regla y el modelo exacto vuelve a asignarlas sin alterar el resto del horario.

Otra posibilidad consiste en combinar una población de horarios con una regla de aceptación propia de la búsqueda local. Abdullah et al. asignan a cada horario una fuerza calculada a partir de su penalización y emplean esa información para dirigir Great Deluge [14]. Como muestra la Fig. 2, las soluciones con menor penalización atraen la búsqueda, las peores la alejan y la combinación produce un nuevo horario.

D. Nuevas líneas de investigación

Una línea reciente adapta el algoritmo a cada instancia y combina construcción exacta con mejora local. De Coster et al. describen cada problema mediante 256 características y entrenan selectores que recomiendan el algoritmo más apropiado para distintas regiones del espacio de instancias [17]. En 2026, Nguyen combina una inicialización CP-SAT con VND y perturbación guiada por penalizaciones sobre ITC-2007 [21]. Estas propuestas desplazan la pregunta desde qué método es mejor en promedio hacia qué método o configuración conviene ante una estructura concreta de conflictos, aulas y disponibilidad.

VI. El vecindario LectureKick

LectureKick es la regla de vecindario con la que el algoritmo genera el siguiente horario. Ceschia et al. la aplican después de una inicialización mediante Hill Climbing y dentro de una fase de enfriamiento simulado [18]. La regla reubica una o dos clases sin incumplir las restricciones obligatorias. Antes de aplicarla, el programa construye un horario factible y calcula las cuatro penalizaciones blandas del problema.

Cada clase ocupa un hueco definido por un periodo y un aula. En la notación de la Fig. 3, l_i identifica una clase, c_i indica el curso al que pertenece y s_i representa un hueco del horario. Por ejemplo, l_2:c_2 significa que la segunda clase pertenece al curso c_2. LectureKick parte de l_1 situada en s_1 y propone trasladarla a s_2. Si el destino está libre, basta con mover l_1. Cuando s_2 contiene otra clase l_2, LectureMove solo permite enviarla al hueco liberado s_1. LectureKick añade la posibilidad de colocar l_2 en cualquier hueco libre s_3. De este modo puede alcanzar horarios que no están disponibles mediante un intercambio directo.

Los destinos deben ser distintos, los periodos deben estar disponibles y las clases implicadas no pueden presentar conflictos. Cuando se utiliza s3, ese hueco debe permanecer libre.

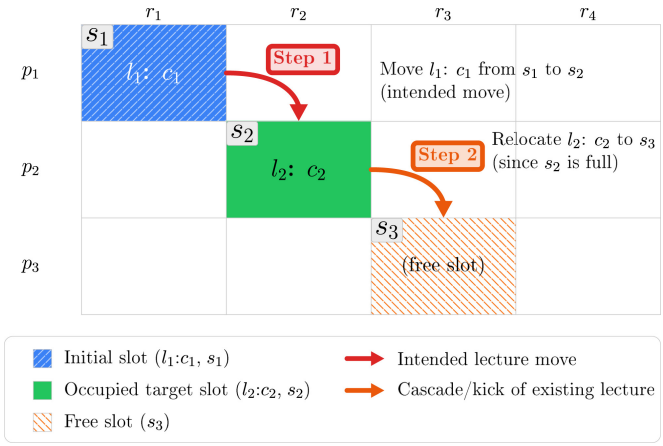


Figura 3. Movimiento LectureKick. El símbolo li identifica una clase, ci su curso y si un hueco formado por periodo y aula. La clase l1 pasa de s1 a s2 y, si el destino está ocupado, l2 puede reubicarse en s3. Reproducido de la Fig. 1 de Ceschia et al. [18].

Estas condiciones hacen que la regla trabaje únicamente con horarios factibles.

La elección de movimientos tampoco es uniforme. Ceschia et al. consideran seis mecanismos de sesgo: días mínimos, cursos críticos vinculados a clases aisladas, capacidad, estabilidad de aula, preferencia por aulas adecuadas y frecuencia de intercambios. Tras el análisis de sensibilidad, el sesgo de capacidad queda fijado a 0 y la preferencia por aulas adecuadas a 1. Los demás parámetros se ajustan junto con el enfriamiento simulado [18]. La Fig. 4 resume estos mecanismos.

Ceschia et al. emplearon *"irace"*, una herramienta de configuración automática que prueba combinaciones de parámetros antes de la evaluación final [18], [19]. El programa comparó distintas combinaciones en un conjunto de entrenamiento y conservó las que ofrecían mejores resultados. Entre los parámetros ajustados estaban la temperatura, la tasa de enfriamiento y las probabilidades de los sesgos activos. La configuración seleccionada se mantuvo después durante las ejecuciones de evaluación.

VII. Metodología de reproducción

Se ejecutó el programa publicado por Ceschia et al. con la configuración utilizada en el artículo [18]. El proceso comienza con Hill Climbing para obtener un horario factible y continúa con el enfriamiento simulado, que utiliza LectureKick para generar horarios vecinos.

A. Diseño experimental

Se seleccionaron las 19 instancias no redundantes de la formulación UD2 de ITC-2007. Se omitió comp11 porque su coste objetivo es nulo y comp15 porque equivale a comp03 bajo esta formulación. Cada instancia se ejecutó tres veces con semillas distintas, lo que dio lugar a 57 ejecuciones. El programa convirtió el límite virtual de 192 segundos en un presupuesto de búsqueda específico para cada instancia. Este límite sirve para fijar el esfuerzo de búsqueda y no representa una medición directa del tiempo de ejecución.

Tabla II
Diferencias técnicas entre el artículo de Ceschia et al. [18] y la reproducción.

Aspecto	Artículo	Reproducción
CPU	Intel i7-7700 (4C/8T)	Intel i7-9700F (8C/8T)
SO	Ubuntu 24.04	Windows 10 Pro
Compilador	G++ 13.3	G++ 16.2 / UCRT64

En cada ejecución, el programa construyó primero un horario y aplicó Hill Climbing hasta alcanzar una solución factible. Después utilizó enfriamiento simulado con LectureKick hasta agotar el presupuesto asignado. Al finalizar se verificaron el código de salida, el coste numérico y la factibilidad del horario.

Por limitaciones computacionales, la reproducción emplea tres semillas por instancia frente a las 30 ejecuciones independientes del artículo y se ejecuta en un entorno diferente. En consecuencia, permite comprobar el procedimiento, la factibilidad y la estabilidad básica de la ejecución, pero no reproducir con la misma potencia estadística las medias y desviaciones publicadas. La Tabla II resume las diferencias técnicas [18], [20].

VIII. Resultados

A. Validez de la campaña reproducida

La campaña produjo 57 ejecuciones válidas con las semillas 0, 1 y 2 y un límite virtual de 192 s. Se utilizaron las versiones fijadas del solver y de EasyLocal++ 3. La configuración reproduce los parámetros finales de Ceschia et al. [18]. El artículo se ejecutó en Ubuntu 24.04 con GNU G++ 13.3 y un Intel Core i7-7700. La reproducción se realizó en Windows 10 con MSYS2 UCRT64, GNU G++ 16.2 e Intel i7-9700F. Por ello, los tiempos se interpretan como evidencia descriptiva y no como una réplica exacta del rendimiento temporal [18], [20].

B. Comparación con el estudio de 2026 y con el estado del arte

Para evitar confundir reproducibilidad con competitividad se emplean dos referencias distintas. Ceschia et al. [18] resuelven cada instancia de validación mediante 30 ejecuciones y, en su Tabla 5, reportan media, desviación estándar, mínimo e iteraciones. Bajo el límite ITC-2007, el enfriamiento simulado obtiene el mejor promedio en 7 de las 19 instancias y el mejor rango medio (1.79) entre los métodos comparados, aunque la prueba de Friedman no distingue globalmente a los métodos al nivel 0.05. Con solo tres semillas, la campaña de este trabajo no permite reproducir esas distribuciones ni repetir sus pruebas estadísticas. Por ello, la comparación con [18] se interpreta como reproducción funcional del protocolo. De forma separada, la Tabla III utiliza los mejores valores conocidos compilados en [2] para medir la distancia al estado del arte, sin atribuir esos valores al algoritmo de 2026.

El mejor resultado reproducido igualó el mejor valor conocido de [2] en 2 de 19 instancias: comp01 (5) y comp13 (59). En las otras 17 instancias el mejor coste fue superior.

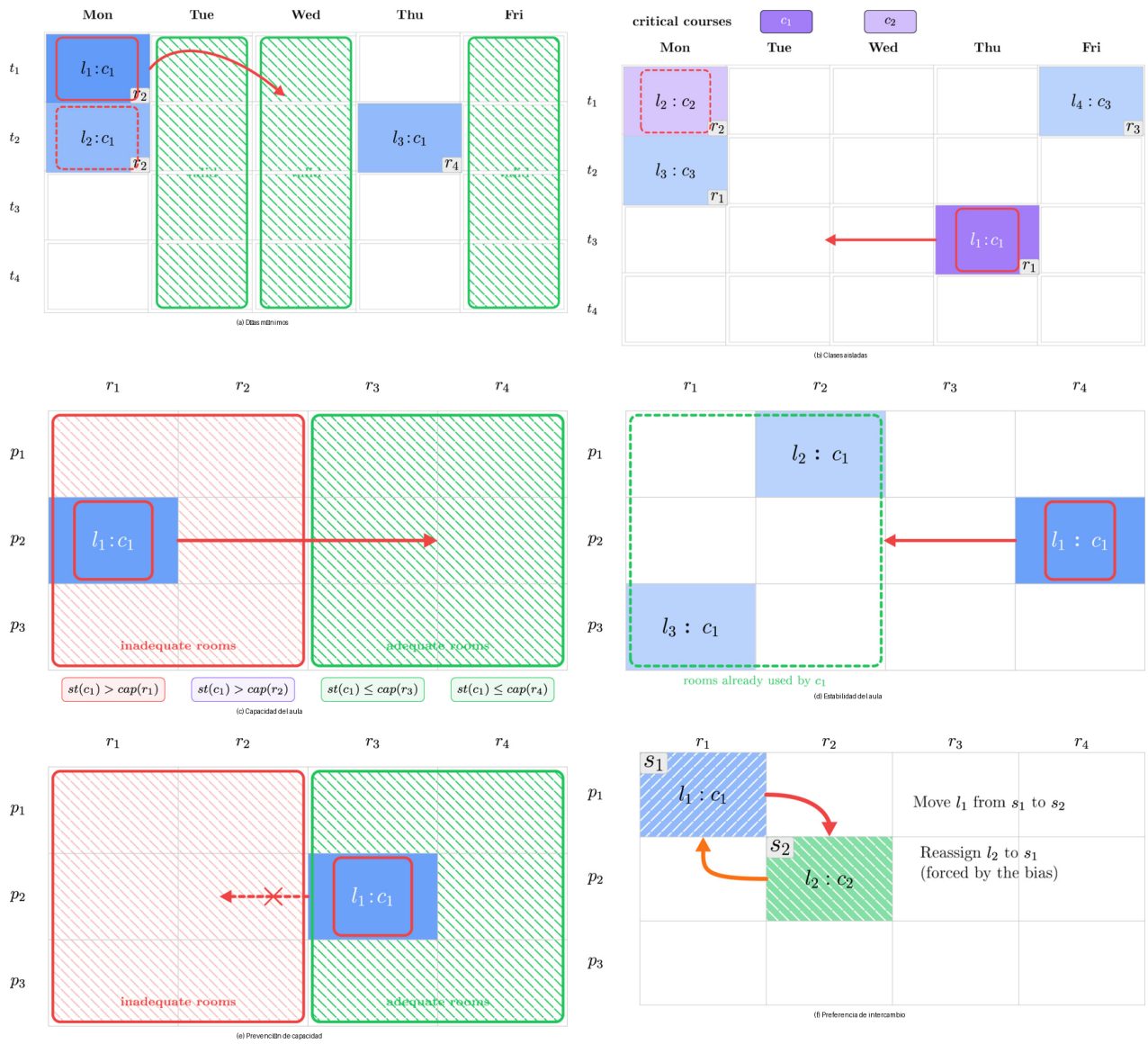


Figura 4. Ejemplos de los seis mecanismos de sesgo considerados en el diseño de LectureKick. En la configuración final, dos quedan fijados tras el análisis de sensibilidad y los restantes se combinan con los parámetros del enfriamiento simulado. Adaptado de la Fig. 2 de Ceschia et al. [18].

El gap porcentual mediano fue 16.2 % y la media fue 46.3 %, valor muy influido por instancias con referencias pequeñas como comp10 y comp20. La media de los mejores costes reproducidos fue 74.68 frente a 66.11 para los mejores valores conocidos. Estas cifras describen la competitividad frente al estado del arte y no equivalen a las 30 ejecuciones del algoritmo de 2026. El tiempo medio reportado por el solver fue 302.5 s por ejecución, pero no es directamente comparable con el tiempo de referencia porque cambia el entorno y el límite virtual se transforma en un número de iteraciones específico por instancia [18].

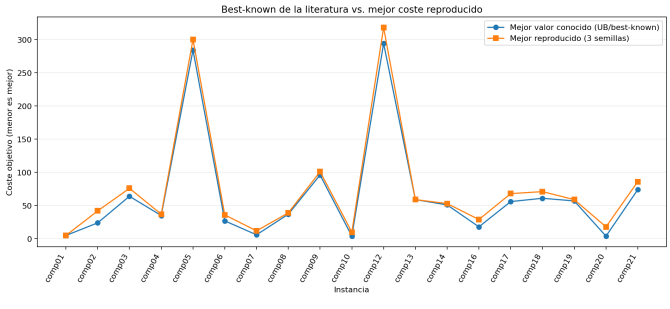


Figura 5. Mejor coste reproducido frente al mejor valor conocido (UB/best-known) de [2] para las 19 instancias no redundantes.

C. Resultados de la revisión cuantitativa

La matriz de extracción contiene 25 estudios primarios publicados entre 2015 y 2026: 21 corresponden al núcleo CB-CTT y 4 a extensiones explícitas. En este conjunto de estudios, los métodos exactos, matemáticos y declarativos constituyen la mayor familia individual (28 %), mientras que búsqueda local, métodos evolutivos y aprendizaje automático, selección

Tabla III
COMPETITIVIDAD DE LA REPRODUCCIÓN FRENTE A LOS MEJORES VALORES CONOCIDOS (UB/BEST-KNOWN) COMPILADOS EN [2]. UN GAP POSITIVO INDICA UN COSTE MAYOR QUE EL BEST-KNOWN.

Inst.	Best-known	Mejor	Media	Gap mejor	t medio (s)
comp01	5	5	5.00	0.0 %	335.2
comp02	24	42	44.00	75.0 %	338.3
comp03	64	76	76.00	18.8 %	278.1
comp04	35	37	38.33	5.7 %	280.5
comp05	284	300	308.33	5.6 %	376.5
comp06	27	36	38.67	33.3 %	326.1
comp07	6	12	13.33	100.0 %	335.1
comp08	37	39	39.67	5.4 %	287.7
comp09	96	101	101.00	5.2 %	295.5
comp10	4	10	12.00	150.0 %	322.1
comp12	294	318	322.67	8.2 %	294.2
comp13	59	59	64.00	0.0 %	276.3
comp14	51	53	53.00	3.9 %	265.8
comp16	18	29	31.33	61.1 %	283.8
comp17	56	68	71.33	21.4 %	292.2
comp18	61	71	71.33	16.4 %	236.8
comp19	57	59	60.00	3.5 %	282.9
comp20	4	18	19.00	350.0 %	350.0
comp21	74	86	94.00	16.2 %	289.7

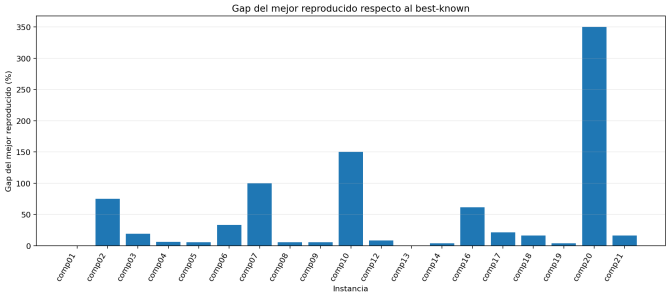


Figura 6. Gap del mejor coste reproducido respecto al mejor valor conocido de [2] por instancia.

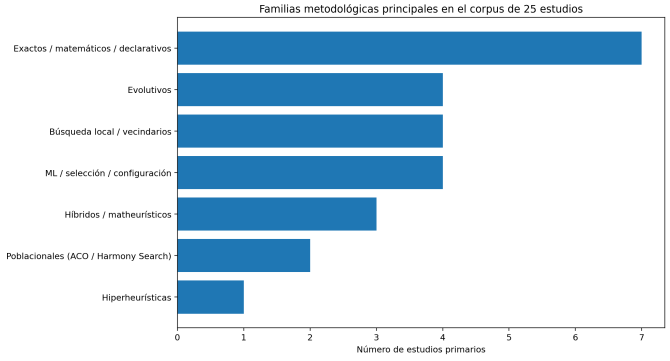


Figura 7. Distribución de las familias metodológicas principales en los 25 estudios primarios.

y configuración representan 16 % cada uno. Dieciséis estudios (64 %) utilizan explícitamente ITC-2007. Veinte (80 %) usan datos públicos o mayoritariamente públicos, pero solo en dos (8 %) se verificó código público junto con una configuración suficientemente detallada para clasificarlos con reproducibilidad alta. La comparación entre estas cifras muestra una diferencia clara entre la disponibilidad del benchmark y la disponibilidad del experimento ejecutable.

IX. DISCUSIÓN Y AMENAZAS A LA VALIDEZ

La reproducción muestra la factibilidad técnica de la implementación de Ceschia et al. [18]: el código compila, las 57 ejecuciones terminan correctamente y las soluciones respetan todas las restricciones duras. Sin embargo, no reproduce la distribución estadística del artículo, que emplea 30 ejecuciones independientes por instancia. Las tres semillas permiten verificar el procedimiento y observar variabilidad básica, pero no establecer equivalencia estadística entre medias y desviaciones. La distancia frente a los mejores valores conocidos de la Tabla III tampoco es un fallo de la implementación, porque esos valores agregan resultados de distintos métodos y presupuestos computacionales [2], mientras que esta campaña utiliza un único protocolo y solo tres semillas.

La principal amenaza a la validez interna procede del cambio de plataforma y del reducido número de repeticiones. Aunque el método reproducido convierte el límite virtual en un presupuesto de iteraciones para reducir la dependencia del hardware, el tiempo medido sigue afectado por el sistema operativo, el compilador, la CPU y la sobrecarga de E/S. Por ello, la comparación debe centrarse en la factibilidad y la calidad de las soluciones, y una réplica estadística del estudio de 2026 tendría que ejecutar 30 repeticiones por instancia y repetir sus pruebas estadísticas [18].

Respecto a la validez externa, ITC-2007 sigue siendo útil porque ofrece una función objetivo y restricciones estandarizadas y aparece explícitamente en el 64 % del conjunto de estudios analizado. Sin embargo, ITC-2019 incorpora student sectioning, estructuras de clases y restricciones institucionales más ricas [5], por lo que un algoritmo competitivo en UD2 no puede considerarse automáticamente adecuado para todas las configuraciones universitarias actuales. Los trabajos recientes amplían la evaluación con otras familias reales, instancias sintéticas y enfoques híbridos o adaptativos [17], [18], [21].

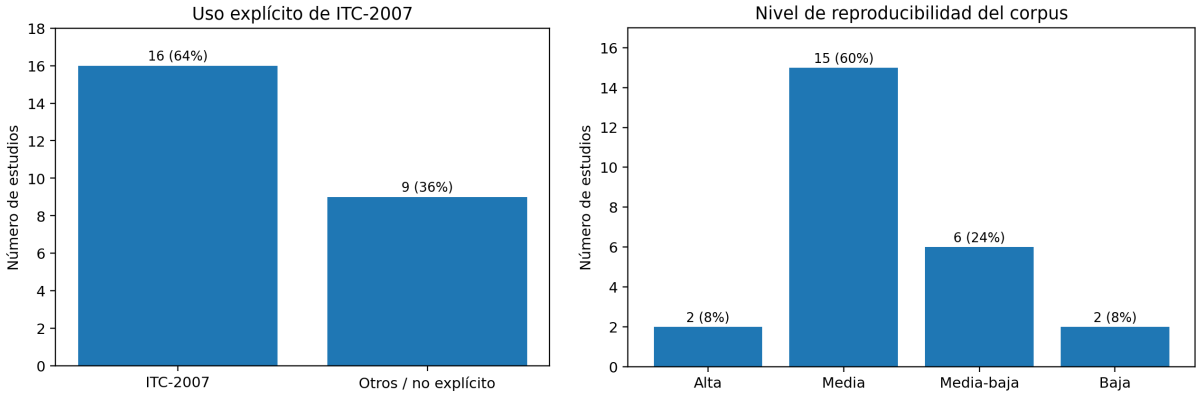


Figura 8. Uso explícito de ITC-2007 y niveles de reproducibilidad en el conjunto de estudios de 25 estudios primarios.

X. Conclusiones

En el conjunto de estudios analizado, CB-CTT muestra una evolución hacia la combinación de optimización exacta, metaheurísticas, hibridación y mecanismos de adaptación. El aprendizaje automático aparece principalmente como apoyo para seleccionar, caracterizar y configurar algoritmos, no como sustituto directo del optimizador [17], [4]. ITC-2007 continúa siendo la referencia común más frecuente, aunque se complementa con conjuntos de datos más diversos para estudiar la generalización y el realismo.

Experimentalmente, se reprodujo de extremo a extremo la implementación basada en LectureKick sobre 19 instancias y tres semillas. Esta reproducción funcional del protocolo de [18] debe distinguirse de la comparación de competitividad: el mejor coste igualó el mejor valor conocido de la literatura en comp01 y comp13, pero permaneció por encima en las demás instancias. Como la campaña utiliza solo tres repeticiones y un entorno distinto, no constituye una réplica estadística completa del artículo de 2026. La revisión muestra una limitación paralela: los benchmarks son mayoritariamente públicos, pero el código y la configuración completa son mucho menos frecuentes.

Referencias

[1] L. Di Gaspero, B. McCollum y A. Schaerf, The Second International Timetabling Competition (ITC-2007): Curriculum-Based Course Timetabling (Track 3), 2007.

[2] S. Ceschia, L. Di Gaspero y A. Schaerf, .Educational timetabling: Problems, benchmarks, and state-of-the-art results,.European Journal of Operational Research, vol. 308, no. 1, pp. 1-18, 2023. doi:10.1016/j.ejor.2022.07.011.

[3] A. Bettinelli, V. Cacchiani, R. Roberti y P. Toth, .An overview of curriculum-based course timetabling,"TOP, vol. 23, pp. 313-349, 2015. doi:10.1007/s11750-015-0366-z.

[4] X. Gu, M. Krish, S. Sohail, S. Thakur, F. Sabrina y Z. Fan, "From Integer Programming to Machine Learning: A Technical Review on Solving University Timetabling Problems,Çomputation, vol. 13, no. 1, 10, 2025. doi:10.3390/computation13010010.

[5] T. Müller, H. Rudová y Z. Müllerová, Real-world university course timetabling at the International Timetabling Competition 2019,"Journal of Scheduling, vol. 28, pp. 247-267, 2025. doi:10.1007/s10951-023-00801-w.

[6] N.-C. F. Bagger, S. Kristiansen, M. Sørensen y T. R. Stidsen, "Flow formulations for curriculum-based course timetabling,.Annals of Operations Research, vol. 280, pp. 121-150, 2019. doi:10.1007/s10479-018-3096-4.

[7] N.-C. F. Bagger, M. Sørensen y T. R. Stidsen, "Dantzig-Wolfe decomposition of the daily course pattern formulation for curriculum-based course timetabling,.European Journal of Operational Research, vol. 272, no. 2, pp. 430-446, 2019. doi:10.1016/j.ejor.2018.06.042.

[8] R. Asín Achá y R. Nieuwenhuis, Çurriculum-based course timetabling with SAT and MaxSAT,.Annals of Operations Research, vol. 218, pp. 71-91, 2014. doi:10.1007/s10479-012-1081-x.

[9] M. Banbara et al., "teaspoon: solving the curriculum-based course timetabling problems with answer set programming,.Annals of Operations Research, vol. 275, pp. 3-37, 2019. doi:10.1007/s10479-018-2757-7.

[10] T. Müller, ÏTC2007 solver description: A hybrid approach,.Annals of Operations Research, vol. 172, no. 1, pp. 429-446, 2009. doi:10.1007/s10479-009-0644-y.

[11] C. Ababei, "Speeding up FPGA placement via partitioning and multithreading,İnternational Journal of Reconfigurable Computing, vol. 2009, pp. 1-9, 2009.

[12] R. Bellio, S. Ceschia, L. Di Gaspero, A. Schaerf y T. Urli, "Feature-based tuning of simulated annealing applied to the curriculum-based course timetabling problem,Çomputers & Operations Research, vol. 65, pp. 83-92, 2016. doi:10.1016/j.cor.2015.07.002.

[13] T. Song et al., Çompetition-guided multi-neighborhood local search algorithm for the university course timetabling problem,.Applied Soft Computing, vol. 110, 107624, 2021. doi:10.1016/j.asoc.2021.107624.

[14] S. Abdullah, H. Turabieh, B. McCollum y P. McMullan, .A hybrid metaheuristic approach to the university course timetabling problem,"Journal of Heuristics, vol. 18, pp. 1-23, 2012. doi:10.1007/s10732-010-9154-y.

[15] S. Abdipoor, R. Yaakob, S. L. Goh y S. Abdullah, "Meta-heuristic approaches for the university course timetabling problem,Ïntelligent Systems with Applications, vol. 19, 200253, 2023. doi:10.1016/j.iswa.2023.200253.

[16] A. Kiefer, R. F. Hartl y A. Schnell, .Adaptive large neighborhood search for the curriculum-based course timetabling problem,.Annals of Operations Research, vol. 252, no. 2, pp. 255-282, 2017. doi:10.1007/s10479-016-2151-2.

[17] A. De Coster, N. Musliu, A. Schaerf, J. Schoisswohl y K. Smith-Miles, .Algorithm selection and instance space analysis for curriculum-based course timetabling,"Journal of Scheduling, vol. 25, no. 1, pp. 35-58, 2022. doi:10.1007/s10951-021-00701-x.

[18] S. Ceschia, F. Da Ros, L. Di Gaspero y A. Schaerf, Revisiting local search for Curriculum-Based Course Timetabling,Çomputers & Operations Research, vol. 195, 107621, 2026. doi:10.1016/j.cor.2026.107621.

[19] M. López-Ibáñez, J. Dubois-Lacoste, L. Pérez Cáceres, T. Stützle y M. Birattari, "The irace package: Iterated Racing for Automatic Algorithm Configuration,.Operations Research Perspectives, vol. 3, pp. 43-58, 2016. doi:10.1016/j.orp.2016.09.002.

[20] L. M. Weber et al., .Essential guidelines for computational method benchmarking,"Genome Biology, vol. 20, art. 125, 2019. doi:10.1186/s13059-019-1738-8.

[21] T. M. Nguyen, .Adaptive Penalty-Guided VND with CP-SAT for Curriculum-Based Course Timetabling,ïn Proc. 18th Int. Conf. on Agents and Artificial Intelligence (ICAART), vol. 4, SciTePress, pp. 3398-3405, 2026. doi:10.5220/0014437700004052.

[22] A. Moris Lara, J. Martin La Peña y J. M. Villacorta Arteaga, “uctp-optimization-ai-review: materiales y recursos de la revisión y reproducción de CB-CTT,’ GitHub, 2026. Disponible: https://github.com/alejandromorislara/uctp-optimization-ai-review